Liquid Types para la verificación de smart contracts con diccionarios

Tomás Jarabroviski (LU XXX/XX, jarabroviskit@gmail.com)
Director: Hernán Melgratti (hmelgra@dc.uba.ar)

25 de septiembre de 2026

Contexto

Smart contracts Los *smart contracts* [Sza97] son programas que permiten realizar transacciones financieras en un entorno distribuido sin la necesidad de contar con autoridades de confianza (*trusted authorities*).

Uno de los principales desafíos en su desarrollo es garantizar su *correctitud*, es decir, que el código se comporte según lo especificado. Los errores o vulnerabilidades en el código pueden tener consecuencias económicas graves, como ha ocurrido en diversos incidentes conocidos. Dado que los contratos inteligentes se ejecutan sobre una *blockchain* —un sistema inmutable—, una vez desplegados no pueden modificarse para corregir fallas. Esto hace que una verificación exhaustiva antes de su implementación sea un paso crucial.

El análisis de los *smart contracts* se aborda tradicionalmente desde dos enfoques: el *estático* y el *dinámico*. El primero se refiere a verificar la correcta implementación del contrato antes de su ejecución, mientras que el segundo se enfoca en garantizar su correcto comportamiento en tiempo de ejecución. En este trabajo nos centramos en la *verificación estática*, la cual busca asegurar de forma preliminar que el código esté libre de vulnerabilidades comunes (como *overflows* o patrones de *reentrancy*) y que cumpla con su especificación.

En [Con25] se desarrolló una herramienta para la verificación estática de *smart contracts* escritos en Solidity. La estrategia empleada consiste en traducir el código fuente de Solidity a Haskell, utilizando las instrucciones `require` y `assert` como base para generar *refinement types*, los cuales actúan como precondiciones y postcondiciones. Estos tipos refinados son procesados posteriormente mediante la herramienta *Liquid Haskell* para realizar la verificación.

Una de las principales limitaciones de dicha propuesta es que no contempla el soporte para estructuras de datos complejos. En particular, una estructura fundamental y de uso común en los contratos inteligentes son los diccionarios o *mappings*. En esta tesis nos proponemos extender el enfoque de [Con25] para incorporar el soporte y la verificación de diccionarios.

Liquid types Los tipos de datos calificados lógicamente (*Logically Qualified Data Types*), o *Liquid Types*, son sistemas de tipos que combinan la inferencia de tipos con la abstracción de predicados para inferir tipos dependientes de forma automática [RKJ08]. Básicamente, un *liquid type* es un tipo de dato acompañado con un *calificador lógico*, esto es, una expresión booleana (predicado) que involucra variables del programa. Los calificadores lógicos permiten construir tipos dependientes que son refinamientos de los tipos estándar, añadiendo restricciones que describen comportamientos específicos y que pueden ser verificadas automáticamente. Por ejemplo, se pueden especificar restricciones sobre los índices de los arreglos, como asegurarse de que un índice esté siempre dentro de los límites del arreglo. Estos refinamientos ayudan a garantizar la corrección del programa.

Los *liquid types* se aplican principalmente en el contexto de lenguajes funcionales, como Haskell, a través de herramientas como LiquidHaskell (LH) [VSJ+14b].

LiquidHaskell es una extensión del sistema de tipos de Haskell que permite refinar los tipos con predicados lógicos. Al agregar anotaciones de tipos liquid en el código, el compilador puede comprobar que se cumplen ciertas propiedades, como la seguridad en el acceso a estructuras de datos indexadas o la preservación de invariantes de tipos. Liquid Haskell se ha aplicado a la verificación de propiedades en distintos dominios [LPR+20, VSJ14a, HVH19].

Los *liquid types* han sido utilizado para la construcción de algunas herramientas de verificación de *smart contracts*. SolType [NSC+22] y Helmholtz [NSC+22] son herramientas de verificación estática que utilizan *liquid types* para verificar propiedades de *smart contracts*. SolType se basa en un sistema de tipos de refinamiento para *Solidity*, diseñado para verificar operaciones aritméticas en contratos inteligentes, en particular, ausencia de overflows. Su enfoque permite expresar relaciones aritméticas complejas y verificar variables enteras que representan activos financieros, como tokens. Helmholtz, por su parte, está diseñado para Michelson, el lenguaje de Tezos. Utiliza tipos de refinamiento para verificar que los contratos escritos en Michelson cumplan con las especificaciones definidas por el usuario.

Objetivo

Extender el enfoque presentado en [Con25] para soportar contratos inteligentes que utilicen estructuras de datos de tipo diccionario.

Actividades

1. Proporcionar una especificación de diccionarios en Liquid Haskell. Las especificaciones existentes en la literatura son débiles, dado que ofrecen pocas garantías (en términos de tipos líquidos) sobre el comportamiento de las traducciones.

2. Analizar manualmente la estrategia para traducir contratos con diccionarios de Solidity a código Haskell, así como su especificación en forma de refinamientos en Liquid Haskell.

3. Extender la implementación del prototipo de traducción de código Solidity a código Haskell desarrollado en [Con25].

4. Evaluar la factibilidad del enfoque propuesto mediante su aplicación en contratos que formen parte de benchmarks conocidos, como por ejemplo, *Azure Blockchain Workbench* o *Rosetta Smart Contracts*.

Duración acordada para la finalización: 4 meses

Referencias

[Con25] Gonzalo Consoli. Liquid types para la verificación de smart contracts. Tesis de licenciatura, Departamento de Computación, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires, septiembre 2025. Director: Dr. Hernán Claudio Melgratti.

[HVH19] Martin AT Handley, Niki Vazou, and Graham Hutton. Liquidate your assets: reasoning about resource usage in liquid haskell. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–27, 2019.

[LPR+20] Yiyun Liu, James Parker, Patrick Redmond, Lindsey Kuper, Michael Hicks, and Niki Vazou. Verifying replicated data types with typeclass refinements in liquid haskell. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–30, 2020.

[NSC+22] Yuki Nishida, Hiromasa Saito, Ran Chen, Akira Kawata, Jun Furuse, Kohei Suenaga, and Atsushi Igarashi. Helmholtz: a verifier for tezos smart contracts based on refinement types. *New Generation Computing*, 40(2):507–540, 2022.

[RKJ08] Patrick M Rondon, Ming Kawaguci, and Ranjit Jhala. Liquid types. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 159–169, 2008.

[Sza97] Nick Szabo. Formalizing and securing relationships on public networks. *First monday*, 1997.

[VSJ14a] Niki Vazou, Eric L Seidel, and Ranjit Jhala. Liquidhaskell: Experience with refinement types in the real world. In *Proceedings of the 2014 ACM SIGPLAN Symposium on Haskell*, pages 39–51, 2014.

[VSJ+14b] Niki Vazou, Eric L Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. Refinement types for haskell. In *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming*, pages 269–282, 2014.